

CPDE Web — Security, Tenancy and Access Plan

Discussion draft · August 2026

1. What standard we build to

GovCon buyers evaluate vendors against a predictable ladder. Build to the whole ladder, certify only as far as contracts require.

Layer	What it is	When
NIST SP 800-171	The control set. 110 controls covering access, audit, config management, IR, media protection.	Now — design against it
SOC 2 Type II	Independent attestation over a 3–12 month window. What a vendor-security review actually asks for.	First paying enterprise client
CMMC Level 2	C3PAO certification of 800-171. Required when a contract flows it down.	When a contract requires it
FedRAMP Moderate	Only if CPDE stores CUI and CDA becomes a cloud service provider. Defer — see §8	

Point to make in the meeting: we design to 800-171 from day one because retrofitting audit logging, access control and configuration management is far more expensive than building with them. We certify when a customer pays for the certification, not before.

2. Platform and data protection

AWS, commercial regions, us-east-1 (where the engine already runs).

Control	Implementation
Encryption at rest	KMS with customer-managed keys, per tenant. RDS, S3, EBS, backups.
Encryption in transit	TLS 1.2 minimum, 1.3 preferred. HSTS. No plaintext internal hops.
Key management	AWS KMS. Separate key per tenant so a key can be revoked without touching others.
Network	Private subnets for database and engine. No public database endpoint. ALB + WAF is the only ingress.
Secrets	AWS Secrets Manager with automatic rotation. No credentials in code, environment variables or repos.
Backups	Automated daily snapshots, 35-day retention, encrypted, cross-region copy. Restore tested quarterly.
Vulnerability management	Container image scanning in ECR, dependency scanning in CI, patch SLA by severity.
Logging	CloudTrail, VPC flow logs, application audit log. Written to a separate account the application cannot delete from.

The last row matters more than it looks. Audit logs an attacker can erase are not audit logs. Ship them to an account with a different trust boundary.

3. Tenant isolation — shared database, row-level security

Decision: multi-tenant shared database. One PostgreSQL instance, every tenant-scoped table carrying `client_id`, isolation enforced by PostgreSQL Row-Level Security. This is how most enterprise SaaS works, Salesforce included, and it is defensible to a GovCon buyer **provided the three controls below are actually in place.**

What this buys

One schema migration instead of one per client. One connection pool. Far simpler operations for a two-person company. Materially lower cost.

What it costs, stated plainly

	Shared database	Database per tenant
Blast radius of one flaw	All tenants	One tenant
Per-tenant key revocation	Not available	Destroy the key
Restore one tenant	Extract and replay rows	Restore the database

Answering "how are we isolated?" Demonstrate RLS coverage One sentence

We accept these. The compensating controls below are what make the answer credible rather than hopeful.

The three non-negotiables

1. `FORCE ROW LEVEL SECURITY`, and the application must not own the tables. A table owner bypasses RLS by default. Application connects as a restricted role that owns nothing. This is the most common way an RLS deployment turns out to be decorative.

```
sql
ALTER TABLE pursuit ENABLE ROW LEVEL SECURITY;
ALTER TABLE pursuit FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON pursuit
USING (client_id = current_setting('app.client_id')::uuid);

-- app_role owns nothing and cannot bypass
GRANT SELECT, INSERT, UPDATE, DELETE ON pursuit TO app_role;
```

2. Tenant context is set per transaction, never per connection. With a pooled connection, `SET` at connect time leaks the previous request's tenant to the next request on that connection. Use `SET LOCAL` inside the transaction, derived from the authenticated session, never from anything the client sends.

```
sql
BEGIN;
SET LOCAL app.client_id = '...'; -- from the auth token, not the request body
-- queries here
COMMIT;
```

3. A cross-tenant test in CI that fails the build. Seed two tenants, authenticate as one, attempt to read the other across every endpoint and every report. One returned row fails the build. This is also the artifact handed to a security reviewer — it converts "we use RLS" into evidence.

Layered on top

Scope predicate on top of tenant isolation. RLS separates companies; `fn_user_visible_org_nodes` separates business units within a company. Two distinct concerns — do not conflate them in one policy.

Encryption at rest via KMS on the RDS instance. Single key, so plan deletion as an operational procedure with evidence, not as key destruction.

Data export and deletion on contract end is a documented, tested runbook producing a certificate of deletion. This replaces the key- destruction answer and clients will ask for it.

Backups are shared. A restore is a row-level operation. Write and test that runbook before a client needs it, not during.

Path to dedicated tenancy stays open. Because everything is keyed on `client_id` and nothing is client-specific in code, moving one client to their own database later is a data migration and a connection-string change — not a rewrite. Price it as a premium option if a client insists.

4. Onboarding a company

Provisioning is a controlled workflow, not a form. CDA runs it.

Create tenant — client record and `client_id`; no new database

Configure org tree — Business → Business Unit → Division as the client defines it

Provision markets — CDA-supplied; differentials stay engine-side

Register email domains — one or more verified domains (§6)

Designate the first admin — named individual, verified out of band (a phone call, not an email), scoped at the top node

Client admin takes over — adds their own users from there

Steps 1–5 are CDA-operated and should be automated as one script with an approval gate, not clicked through a console. Step 6 is self-service.

Deprovisioning needs the same rigor and usually gets none. Contract ends: disable access immediately, export their data in a documented format, retain per contract, then delete their rows and issue a certificate of deletion. With a shared database this is a tested runbook, not a key destruction — write it before a client asks.

5. Admin delegation

Roles, narrowest to broadest:

Role	Can do
Read Only	View pursuits within assigned scope
Capture Manager	Create and edit pursuits, answer Pwin questionnaires, within scope
BU Admin	The above, plus add and remove users at their node and below
Client Admin	All nodes for that client, plus org structure and targets
CDA Support	Cross-tenant, break-glass only, time-boxed, every action logged and reviewed

The delegation rule is the same recursive scope predicate that governs pursuit visibility: **an admin can only grant access to nodes at or below their own, and can never grant a role above their own.** Both halves matter — without the second, a BU Admin can create a Client Admin and escalate.

CDA Support access is the one auditors will ask about. Time-boxed elevation, business justification recorded, notification to the client admin, automatic expiry. Not a standing account.

6. Email domain verification

Every client registers one or more domains. A user can only be invited at an address matching a registered domain for that client.

This will not survive contact with reality, and it is better to say so now:

Contractors and consultants on a capture team often carry their own domains. Real requirement, immediate exception.

Subsidiaries and acquisitions — a large primes' BUs frequently use different domains.

Joint ventures — a JV pursuit may legitimately involve two companies.

Personal-domain small firms exist in this market.

So: domain matching as the **default**, plus an explicit exception list where a Client Admin can approve a named address, with a reason, recorded in the audit log. The control is not that exceptions are impossible — it is that exceptions are deliberate, attributable and reviewable.

7. Authentication

SSO wherever the client can support it. This is strongly preferred and should be the opening position.

SAML 2.0 or OIDC federation to the client's IdP (Entra ID, Okta, Ping)

Their password policy, their MFA, their conditional access

Offboarding is automatic — when their directory account dies, so does CPDE access. We never have to be told. This is the single strongest security argument in the plan.

Per-client IdP configuration: entity ID, certificate, metadata URL, attribute mapping. Certificates expire — track and alert.

SCIM provisioning is a separate protocol and a separate effort. SSO handles login; SCIM handles account create/update/disable. Automatic offboarding technically requires SCIM. Scope them as two phases and do not promise the second while delivering the first.

Fallback for clients who cannot federate: managed identity provider (AWS Cognito or Auth0) — never home-grown.

MFA mandatory, no opt-out. TOTP authenticator app minimum.

Not SMS. NIST deprecated SMS as an out-of-band authenticator; a GovCon security reviewer will flag it.

Invitation flow: admin invites → signed single-use token, short expiry → user sets password and enrolls MFA before first access

Session timeout, re-authentication for sensitive actions, account lockout and password policy all owned by the IdP, not by us

We do not store passwords. Ever. No password column exists in the schema and none should be added.

Handling credentials directly is the fastest route to failing a SOC 2 audit.

8. What we are deliberately NOT doing yet

FedRAMP. Only required if CPDE stores CUI, which today it does not — the data is the client's own internally-generated BD material. If we ingest win/loss debriefs or RFP attachments, that changes and CDA becomes a cloud service provider subject to FedRAMP Moderate or 3PAO-assessed equivalency. Running on GovCloud does not confer this on our application. **Six-figure decision. Wait for a contract that requires it.**

GovCloud. Follows the same trigger.

Client-hosted deployment. Kills our release cadence. Priced accordingly if a client insists.

Design consequence we accept now: documents (debriefs, RFPs) go to object storage with a `document` table holding metadata and a classification tag — never as database blobs. That is what makes a future CUI enclave a migration rather than a rewrite.

9. Sequence

Phase 1 — before first client RLS with FORCE and a non-owning application role, per-transaction tenant context, cross-tenant CI test, audit logging to a separate account, SSO via managed IdP, MFA on the fallback path, admin delegation, domain verification with exceptions, tested export-and-delete runbook.

Phase 2 — first paying enterprise client SOC 2 Type II readiness and audit. Formal IR plan. Backup restore testing. Vulnerability management SLA. Vendor security questionnaire responses.

Phase 3 — contract-driven SCIM. CMMC L2 if flowed down. FedRAMP only if we hold CUI.

10. Open questions for the discussion

Do clients want to manage CPDE roles from their own directory groups, or manage them inside CPDE with SSO only for authentication? Enterprise IT usually wants the first; it is more work for us and less for them.

Will any target client refuse a shared database, regardless of RLS? Some enterprise security teams have a hard rule. Determines whether dedicated tenancy is theoretical or immediate — and it is a price conversation, not an architecture one, because the migration path is already open.

Who at CDA holds the security role? SOC 2 requires a named owner for access reviews, incident response and vendor management. With two people this is a real constraint, not a formality.

Do we commit to a published SLA? Enterprise contracts will ask. Availability targets carry architecture cost.

Unified API vendor for CRM import becomes a subprocessor holding client pipeline data. Check FedRAMP status and SOC 2 before standardizing on one.

This plan reflects design intent, not implemented state. Nothing in sections 2–7 is built today; the schema supports it and the decisions are made. I am not a security assessor — an hour with a C3PAO or a SOC 2 auditor before committing architecture is worth the cost.